

CS & IT ENGINEERING



Data Structure & Programming

Linked Lists

Lecture No.- 02



By- Satya sir

Recap of Previous Lecture



- Array - Insertion
 - Deletion
 - LTM, UTM
- Linked Lists Intro



Topics to be Covered



- Insertion
 - Construction
 - Deletion
 - Search
- } SLL



Topic : SLL – Creation, Insertion



Element of LL == Node

- Self referential structure Variables must be Pointer Variables.

Singly Linked List

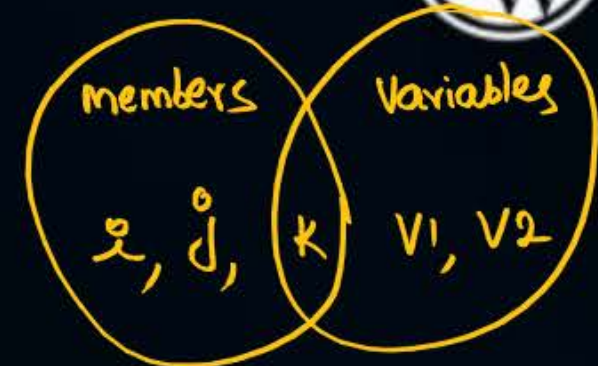
Each node must have 2 Parts

- 1) Data : Any type, Any amount
- 2) Link : Only single.

Ex: Struct A

```

{
    int i;
    float j;
    struct A *k;
} *v1, *v2;
```



Creation of Node : Self-referential structure : A structure whose member/variable is variable member of itself

SLL Node :

Data	Link

```

Struct SLL
{
    datatype data;
    struct SLL *link;
};
```

```

Struct SLL
{
    int Number;
    char Name[10];
    struct SLL *next;
} *head = NULL;
```

Struct SLL *N1, *N2, *N3, *N4;

N1 → Number = 1
 N1 → Name = 'Radha'
 N1 → next = N2

N2

N3

N4



Topic : SLL – Creation, Insertion



Construction of SLL $\cong$ Insertion of a Node

- Construction is Equal to Series of Cumulative Insertion operations.

Let, The list to be constructed is $\boxed{A} \rightarrow \boxed{B} \rightarrow \boxed{C} \rightarrow \boxed{D} \rightarrow \boxed{E} \rightarrow \text{NULL}$

Insertion Sequence $\cong$ Insertion at Beginning
E, D, C, B, A

$\boxed{E} \rightarrow \text{NULL}$

$\boxed{D} \rightarrow \boxed{E} \rightarrow \text{NULL}$

$\boxed{C} \rightarrow \boxed{D} \rightarrow \boxed{E} \rightarrow \text{NULL}$

$\boxed{B} \rightarrow \boxed{C} \rightarrow \boxed{D} \rightarrow \boxed{E} \rightarrow \text{NULL}$

$\boxed{A} \rightarrow \boxed{B} \rightarrow \boxed{C} \rightarrow \boxed{D} \rightarrow \boxed{E} \rightarrow \text{NULL}$

Insertion Sequence $\cong$ Insertion at the End
A, B, C, D, E

$\boxed{A} \rightarrow \text{NULL}$

$\boxed{A} \rightarrow \boxed{B} \rightarrow \text{NULL}$

$\boxed{A} \rightarrow \boxed{B} \rightarrow \boxed{C} \rightarrow \text{NULL}$

$\boxed{A} \rightarrow \boxed{B} \rightarrow \boxed{C} \rightarrow \boxed{D} \rightarrow \text{NULL}$

$\boxed{A} \rightarrow \boxed{B} \rightarrow \boxed{C} \rightarrow \boxed{D} \rightarrow \boxed{E} \rightarrow \text{NULL}$

Insertion sequence $\cong$ Insertion at middle
E, A, D, B, C
[Neither first nor last]

$\boxed{E} \rightarrow \text{NULL}$

$\boxed{A} \rightarrow \boxed{E} \rightarrow \text{NULL}$

$\boxed{A} \rightarrow \boxed{D} \rightarrow \boxed{E} \rightarrow \text{NULL}$

$\boxed{A} \rightarrow \boxed{B} \rightarrow \boxed{D} \rightarrow \boxed{E} \rightarrow \text{NULL}$

$\boxed{A} \rightarrow \boxed{B} \rightarrow \boxed{C} \rightarrow \boxed{D} \rightarrow \boxed{E} \rightarrow \text{NULL}$



Topic : SLL – Creation, Insertion



Insertion / Add a Node

- 1) At Beginning
- 2) At End
- 3) At given Position (Neither first, Nor last)

Insertion at Beginning

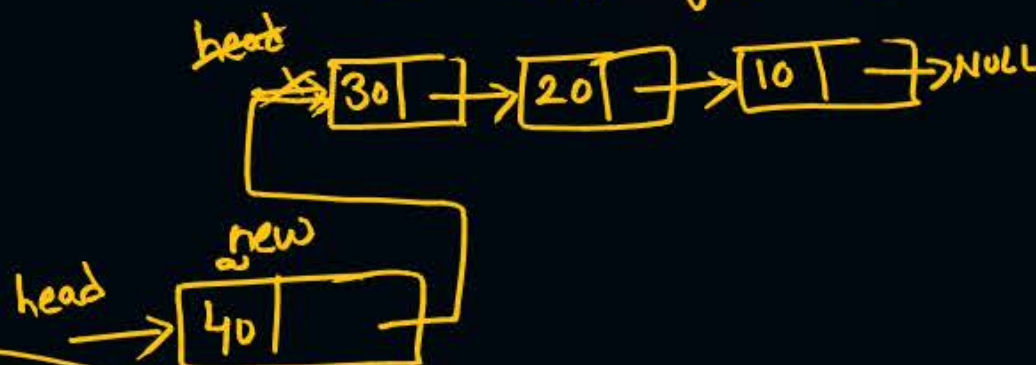
```
Void Insert_Beginning_SLL(int value)
{
    Struct SLL *new;
    new = (Struct *) malloc(sizeof...);
    new->a = value;
    if (head == NULL)
    {
        head = new;
        new->next = NULL;
    }
    else
    {
        new->next = head;
        head = new;
    }
}
```

Time Complexity: $O(1)$

Struct SLL

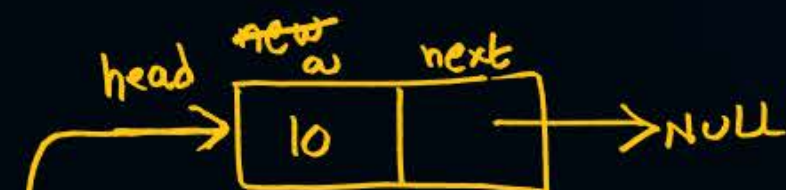
```
{
    int a;
    Struct SLL *next;
} *head = NULL;
```

Insert_Beginning_SLL(40);

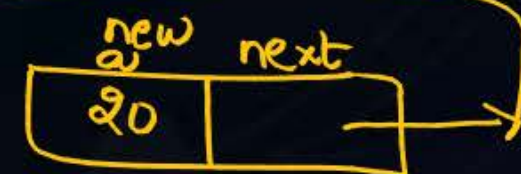


for (i=10; i<=50; i=i+10)
Insert_Beginning_SLL(i);

Insert_Beginning_SLL(10);



Insert_Beginning_SLL(20);



Insert_At_Beginning_SLL(30);





Topic : SLL – Creation, Insertion



Insertion at End

Time Complexity = $O(n)$

```
void Insert_End_SLL(int Value)
```

```
{
```

```
    struct SLL *new;
```

```
    new = (struct *) malloc (sizeof (struct SLL *));
```

```
    new->data = Value;
```

```
    if (head == NULL) // Empty list
```

```
    {
```

```
        head = new;
```

```
        new->next = NULL;
```

```
    }  
    else
```

```
    { struct SLL *temp = head;
```

```
        while (temp->next != NULL)
```

```
        { temp = temp->next;
```

```
        }
```

```
temp->next = New;
```

```
new->next = NULL;
```

```
}
```

```
}
```

Empty list

$\Rightarrow$ head = NULL

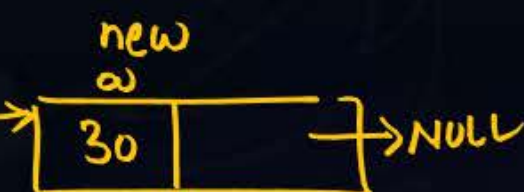
Insert_End_SLL(50);



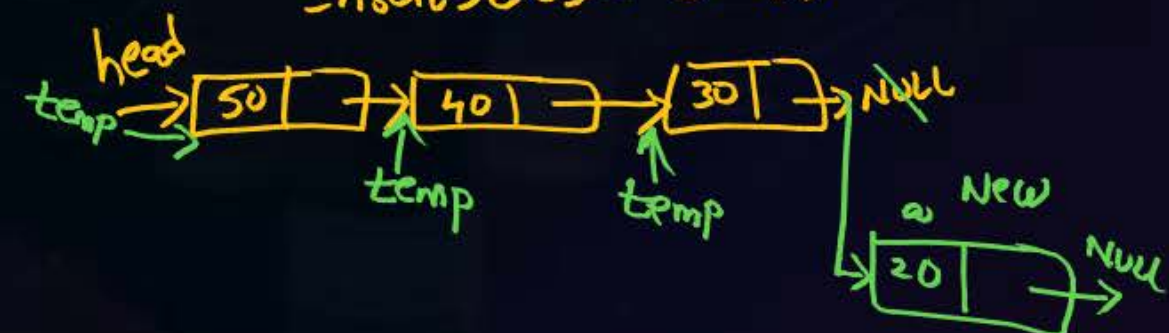
Insert_End_SLL(40);



Insert_End_SLL(30);



Insert_End_SLL(20);





Topic : SLL – Deletion, Search



Insertion at middle / At given Position

(Assume Position $\neq 1$ and not n)

Void Insert_middle_SLL (int value, int Position)
// 50 // 4

{

Struct SLL *new;

new = (Struct *) malloc (sizeof (Struct SLL *));

new->data = value;

if (head == NULL)

{
head = new;
new->next = NULL;
}

else

{ int count = 1;

Struct SLL *temp = head;

while (count < Position - 1)

{ temp = temp->next;

count++;
}

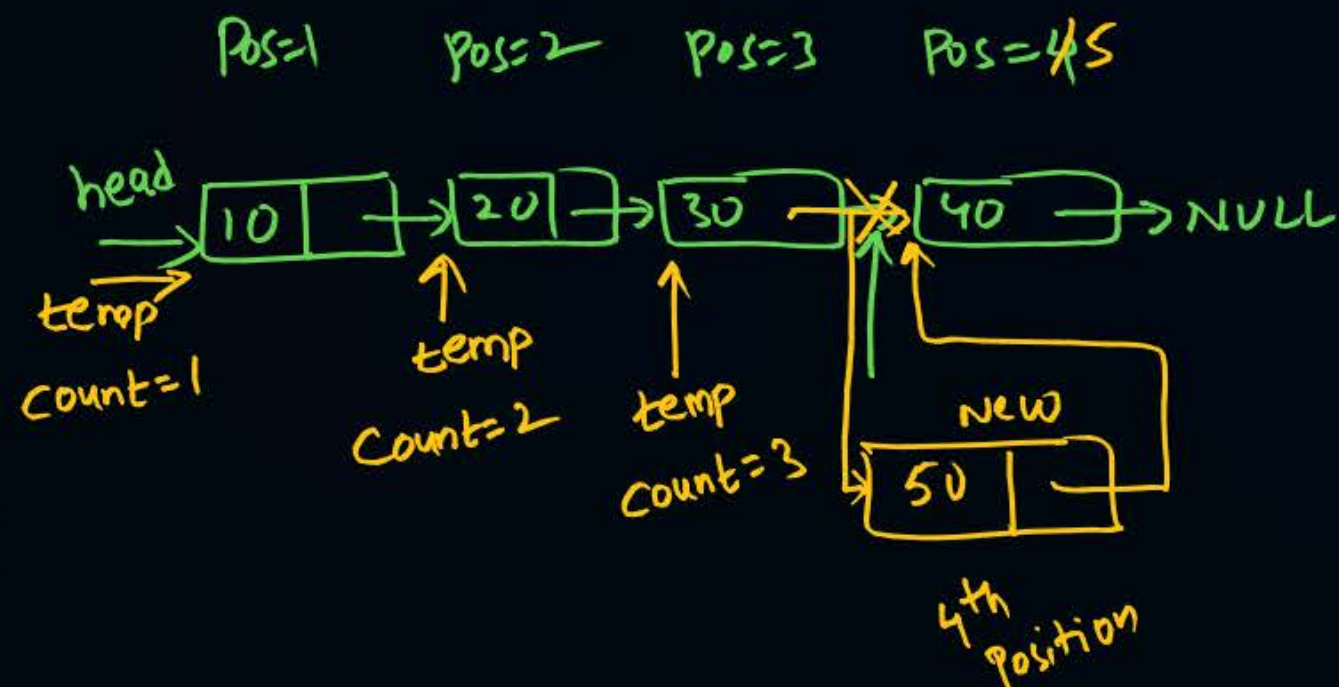
new->next = temp->next;

temp->next = new;

}

}

Time Complexity:
 $O(n-1) \cong O(n)$





Topic : SLL – Deletion, Search



Deletion :

- 1) First Node
- 2) Last Node
- 3) middle Node

```
void Delete_first Node( )
```

```
{  
    if (head == NULL)  
    { printf("Empty list");  
    }
```

```
else
```

```
{ struct SLL *temp = head;
```

```
    head = head->next;
```

```
    printf("Deleted Element is %d", temp->data);
```

```
    free(temp);
```

```
} }
```



Time Complexity:

$O(1)$



Topic : SLL – Deletion, Search



Deletion of last Node Code } Home work
Deletion of middle Node code }



2 mins Summary

- SLL Creation
- SLL Construction
- SLL Insertion
 - At Front
 - At End
 - At middle
- Deletion - front Node

To be Contd... 

THANK - YOU